

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

NAME:K.DEEPIKA

REG NO:192371042

SUBJECT:INTERNET PROGRAMMING

1. Servlet Architecture & Workflow

Introduction

A **Java Servlet** is a server-side Java program that receives client requests, processes them, interacts with databases or other services, and sends responses back to the client.

Servlet Architecture

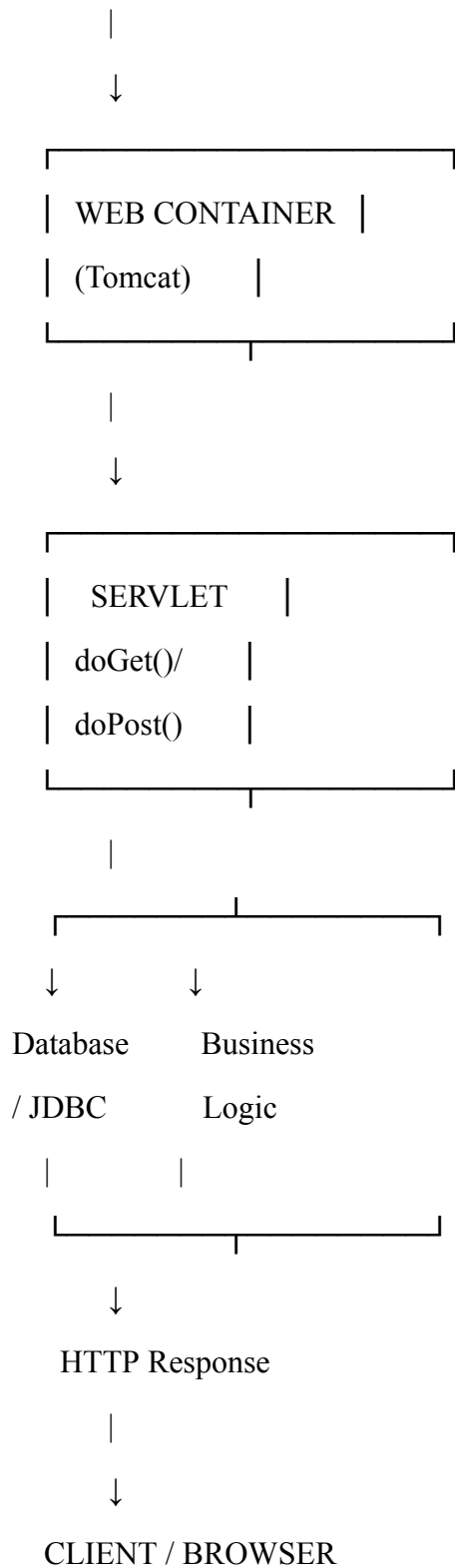
CLIENT / BROWSER

|

| HTTP Request

↓





Request–Response Cycle

1. The client sends an HTTP request.
2. The web server receives the request.
3. The **Web Container** identifies the appropriate Servlet.

4. The container creates `HttpServletRequest` and `HttpServletResponse` objects.
5. The Servlet processes the request.
6. It may communicate with a database using JDBC.
7. The Servlet generates a response.
8. The container sends the HTTP response back to the client.

Role of Web Container

The Web Container, such as **Apache Tomcat**, is responsible for:

- Managing Servlet life cycle
- Creating Servlet objects
- Handling HTTP requests and responses
- Managing multiple client requests
- Providing security and session management
- Calling `init()`, `service()`, and `destroy()`

Request and Response Objects

HttpServletRequest contains information received from the client.

Examples:

```
request.getParameter("username");  
request.getMethod();  
request.getSession();
```

HttpServletResponse is used to send information back to the client.

Examples:

```
response.setContentType("text/html");  
response.getWriter();
```

Real-Time Example

In an e-commerce application:

User clicks "Buy Now"

↓

HTTP Request

↓

Servlet



Check product



Database



Process order



HTTP Response



"Order Placed Successfully"

2. Servlet Life Cycle Analysis

The Servlet life cycle describes how a Servlet is **created, initialized, used, and destroyed** by the Web Container.

Sequence Diagram

Web Container



| Create Servlet



Servlet Object



| init()



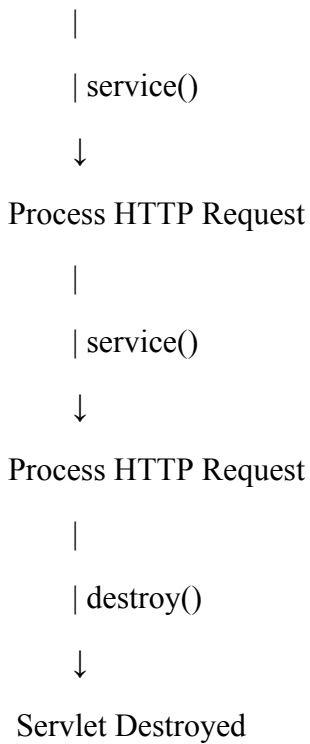
Initialized Servlet



| service()



Process HTTP Request



Three Important Methods

1. init()

Called **only once** when the Servlet is initialized.

```
public void init() {  
    // Initialization  
}
```

Used for:

- Opening database connections
- Loading configuration
- Initializing resources

2. service()

Called for **each client request**.

```
protected void service(  
    HttpServletRequest request,  
    HttpServletResponse response) {  
  
    // Request processing
```

```
}
```

It determines whether the request is GET, POST, etc., and normally dispatches to methods such as `doGet()` and `doPost()`.

3. `destroy()`

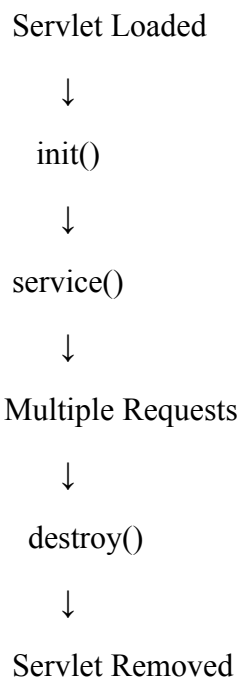
Called **once** before the Servlet is removed.

```
public void destroy() {  
    // Cleanup  
}
```

Used for:

- Closing database connections
- Releasing resources
- Performing cleanup operations

Life Cycle



Performance Implications

- `init()` runs only once, so expensive initialization can be performed there.
- `service()` runs for every request, so it should be efficient.
- `destroy()` ensures resources are properly released.
- Usually, one Servlet instance handles multiple requests concurrently, so shared mutable data must be handled carefully.

3. GET vs POST Method Evaluation

GET and POST are HTTP methods commonly used for form submission.

Feature	GET	POST
Data location	URL	Request body
Visibility	Visible in URL	Not normally visible in URL
Data size	Limited by URL/browser/server limits	Larger payloads supported
Bookmarking	Possible	Not normally used
Caching	Can be cached	Generally not cached by default
Sensitive data	Not appropriate	Better choice
Typical use	Search/retrieval	Login, registration, submission

GET Example

```
<form action="SearchServlet" method="get">
  <input type="text" name="query">
  <button type="submit">Search</button>
</form>
```

URL might become:

/search?query=laptop

Appropriate GET Scenarios

- Search
- Filtering products
- Viewing information
- Reading public resources
- Shareable/bookmarkable URLs

POST Example

```
<form action="LoginServlet" method="post">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="submit">Login</button>
```

</form>

Appropriate POST Scenarios

- Login
- Registration
- Payment/order submission
- Uploading data
- Creating or modifying server-side resources

Security Point

POST is **not automatically encrypted**. For sensitive information, the application should use **HTTPS**. GET is particularly unsuitable for passwords or other sensitive values because parameters appear in the URL and may be recorded in browser history, logs, bookmarks, or analytics systems.

4. Session Tracking Techniques

HTTP is **stateless**, so the server needs session-tracking mechanisms to remember a user between requests.

Three important techniques are:

1. HttpSession
 2. Cookies
 3. URL Rewriting
-

A. HttpSession

HttpSession stores user-specific data on the **server**.

```
HttpSession session = request.getSession();
```

```
session.setAttribute("username", "Aswitha");
```

Retrieve:

```
String name =
```

```
(String) session.getAttribute("username");
```

Advantages

- Data is maintained on the server.
- Suitable for login information and shopping carts.
- Easy to implement.
- Better suited for sensitive session state than storing the state itself in cookies.

Limitations

- Uses server memory/resources.
 - Session expires after inactivity or invalidation.
 - Session management is required across multiple servers in distributed systems.
-

B. Cookies

Cookies store small pieces of information in the **client's browser**.

Cookie cookie =

```
new Cookie("username", "Aswitha");
```

```
response.addCookie(cookie);
```

Advantages

- Lightweight
- Can persist across browser sessions
- Useful for preferences
- Reduces need for server-side storage for small non-sensitive data

Limitations

- Users can disable or delete cookies.
 - Limited storage capacity.
 - Client-side data can be modified.
 - Sensitive information should not be stored directly in cookies.
-

C. URL Rewriting

When cookies are unavailable, the session ID can be added to the URL.

Example:

```
profile;jsessionid=ABC123
```

In Servlets:

```
String url =
```

```
    response.encodeURL("profile");
```

Advantages

- Works even when cookies are disabled.
- Can maintain session continuity.

Limitations

- Session IDs may appear in URLs.
- URLs can be copied or shared accidentally.
- Less convenient and less secure than normal cookie-based session identification.

Comparison

Technique	Storage	Main Advantage	Main Limitation
HttpSession	Server	Secure server-side state	Uses server resources
Cookies	Client	Simple and persistent	Can be disabled/modified
URL Rewriting	URL	Works without cookies	Session ID exposed in URL

5. Cookies vs Sessions – Critical Analysis

Basic Difference

COOKIES

Browser

|
└─ Stores data

|
↓

Server

SESSION

Browser

|
└─ Stores Session ID

|

↓

Server

|

└─ Stores actual session data

Comparison

Feature	Cookies	Sessions
Storage location	Client	Server
Data stored	Small pieces of data	User/session state
Security	More exposed to client-side attacks/modification	Actual data remains server-side
Server memory	Low	Higher
Lifetime	Session or persistent	Usually timeout-based
Size	Limited	Depends on server resources
User can disable?	Yes	Session tracking may depend on cookies/URL rewriting
Best use	Preferences, identifiers	Login, shopping cart

Real-Time Use Cases

Cookies:

Online shopping website

↓

User selects dark mode



Preference stored in cookie



User returns later



Dark mode is restored

Session:

User logs into Amazon-like website



Server creates session



Session stores user/cart information



User visits multiple pages



Cart remains available

Security Considerations

Cookies should use appropriate security attributes such as:

- Secure — sends the cookie over HTTPS.
- HttpOnly — prevents normal JavaScript access to the cookie.
- SameSite — helps control cross-site cookie sending.

Sessions should use:

- HTTPS
- Secure session identifiers
- Appropriate session timeout
- Session invalidation after logout
- Session-ID regeneration where appropriate

Final Conclusion

Cookies are best for small client-side information such as preferences, while sessions are better for maintaining server-side user state such as authentication and shopping carts. In many Java web applications, an HttpSession is identified by a JSESSIONID cookie, so the two mechanisms can work together rather than being completely separate.